SYRAX: A VOICE-DRIVEN 3D PARTICLE WEB UI AGENT

A Project Synopsis

Submitted by

Ayush Kumar Upadhyay
2500350100012

Navam
2500350100041

Sandeep
2500350100057

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING



At

BABU BANARSI DAS INSTITUTE OF TECHNOLOGY, GHAZIABAD
Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow
Department of CSE

OCTOBER 2026

Introduction

Syrax is a voice-driven web agent that allows a person to control a real web browser entirely by speaking. Delivered as an installable progressive web application (PWA), it runs from a single web address on both laptop and Android phone. The user taps the microphone once and gives one clear spoken command — for example, “open YouTube”, “search for cats”, “play any comedy video” or “what is the world clock” — and the system either performs the action in a genuine browser tab or answers the question aloud. Unlike conventional chat assistants that merely describe what they would do, Syrax executes the operation and leaves the resulting page open and fully under the user’s control.

Technically, the system is built around three cooperating layers. The presentation layer is a three-dimensional particle interface rendered with Three.js and React Three Fiber, which reacts to microphone level and to application state so that feedback is always visible without obstructing the conversation. The voice layer captures speech through the Web Speech API (speech-to-text, STT), buffers it until the sentence is demonstrably complete, and speaks replies through a layered text-to-speech (TTS) chain that falls back gracefully from a neural voice service to the browser’s built-in synthesiser. The decision layer consists of a deterministic local intent engine that instantaneously matches clear commands, and a large language model (LLM) — DeepSeek’s deepseek-flash — that answers questions and holds conversation but is structurally forbidden from narrating actions it cannot perform.

The principal technologies employed are React 19 with TypeScript, built using Vite; Tailwind CSS for styling; Three.js, React Three Fiber and GSAP for graphics and animation; the Web Speech API for recognition; the DeepSeek application programming interface (API) in JSON mode for natural-language understanding; a small FastAPI sidecar in Python for optional voice services; and GitHub Actions with GitHub Pages for continuous integration and free

hosting. No server-side database is required; conversation history and settings are held in the browser's localStorage.

The project lies in the field of human–computer interaction (HCI) and applied artificial intelligence. Specialised terms used throughout include STT and TTS (speech-to-text and text-to-speech), intent parsing (classifying an utterance into an executable command), LLM and JSON mode (structured model output), PWA (installable web application), one-shot microphone control (the microphone enables itself for a single utterance and always returns to the off state after each task), and popup-safe tab operation.

The objectives of the project are: (i) to execute every clear spoken command reliably without falling back to conversation, (ii) to answer identity, time, date and general questions instantly and honestly, (iii) to perform all operations in real, visible browser tabs, (iv) to keep the microphone privacy-aware — one-shot and never hearing media audio, and (v) to deploy the complete system at zero hosting cost from a public source repository.

Feasibility Study

Technical feasibility: the entire system is realised with browser-native, standards-based technologies — Web Speech, Web Storage, window management — and mature open-source frameworks (React, Vite, Tailwind, Three.js). No bespoke hardware, custom server or proprietary licence is required, and the speech recognition and synthesis services are supplied free of charge by the browser vendor; therefore the proposed work is technically achievable with the available laboratory resources.

Operational feasibility: a single microphone tap followed by one natural sentence is the complete interaction model, so the system requires minimal training. Every action opens in a normal browser tab that the user can inspect, edit or close, which keeps the agent transparent in daily use and installs on desktop and Android from the same link.

Economic feasibility: hosting on GitHub Pages is free, the speech interfaces cost nothing, and the language-model key is optional — without it, all local commands still execute and questions receive canned honest replies. When a key is supplied, usage is pay-per-use at a negligible cost. Overall project cost is therefore close to zero.

Schedule feasibility: the work decomposes into eight well-bounded phases (Table 1) that fit comfortably within the academic project timeline ending October 2026.

Need and significance: hands-free operation improves accessibility for users who cannot type, removes the overhead of menus and navigation for routine tasks such as opening, searching and media playback, and demonstrates that a lightweight, honest assistant can be built and deployed by students on open tooling — a reusable foundation for further research in conversational web agents.

Methodology/Planning of work

The project follows the software development life cycle (SDLC) in eight sequential phases, each concluding with a review of demonstrable, working software. Requirements were gathered first-hand from intended usage scenarios — opening sites, searching, media playback, conversation and halting — and each requirement was classified as either "executable locally" or "requires the language model", which became the central design constraint of the system.

Table 1: Project development phases and duration

S. No.	Phase	Key activities	Duration
1	Requirement analysis	Usage scenarios, command inventory, risk list	2 weeks
2	System design	Layered architecture, module interfaces, theme palette	1 week
3	UI and 3D theme	React shell, Tailwind layout, particle field, PWA assets	2 weeks
4	Voice pipeline	STT buffering, silence/dangling	2 weeks

		detection, TTS chain	
5	Intent engine and LLM	Local command parser, DeepSeek JSON integration	2 weeks
6	Browser operations	Tab reuse, popup-safe opening, video resolution	1 week
7	Testing and hardening	Type checks, live command suite, edge cases	1 week
8	Deployment and documentation	Actions pipeline, Pages hosting, reports	1 week

Design: the architecture is a strict layered pipeline — microphone → STT buffer → local intent gate → either a tab operation or the LLM → text-to-speech reply. The intent gate is deterministic and runs first; the language model is only reachable for content the gate declines (questions and conversation) and holds no actuator at all, so a fabricated "opening…" reply is impossible by construction.

Implementation: modules were built bottom-up and integrated continuously — interface and theming first, then the voice pipeline, then the intent engine, then real-tab operations, and finally the language-model conversation layer. Every merged increment was compiled with strict TypeScript checks before it was accepted.

Testing: the strategy combines static checking (tsc, zero-error policy), build verification (production Vite bundle), and an end-to-end command suite executed against the live deployment — each documented voice phrase is issued and the resulting tab URL, status text and reply are asserted. Edge cases covered include cut-off sentences, misheard wake words, popup blocking, resolver failure and missing API keys.

Deployment: pushes to the main branch trigger a GitHub Actions workflow that builds the application and publishes it to GitHub Pages under a hashed bundle address; a single refresh loads the new version. Maintenance consists of monitoring command success, fixing defects, and extending the command vocabulary in the local parser.

Facilities required for proposed work

Software required:

- Operating system: Windows 10/11 (or any desktop OS with a Chromium browser)
- Browser: Google Chrome / Microsoft Edge (Web Speech API support)
- Editor: Visual Studio Code; runtime: Node.js 18 or later; Git for version control
- Frameworks and libraries: React 19, TypeScript 5.7, Vite 6, Tailwind CSS 4, Three.js, React Three Fiber, GSAP
- Python 3.12 with FastAPI, Uvicorn (optional local voice sidecar)
- DeepSeek API account (optional key, entered once per device in the application)
- GitHub account with GitHub Actions and GitHub Pages (free tier)

Hardware required:

- Any personal computer with at least 4 GB RAM and a microphone input
- Headset or desktop microphone and speakers for voice input/output
- Internet connection (for speech services, the language model and hosting)
- An Android smartphone (optional) for verifying the installed progressive web app

Bibliography

[1] MDN Web Docs, “Web Speech API”, developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API (Accessed: October 2026).

[2] MDN Web Docs, “Window.open()”, developer.mozilla.org/en-US/docs/Web/API/Window/open (Accessed: October 2026).

[3] React Foundation, “React documentation”, react.dev/learn (Accessed: October 2026).

[4] Three.js, “Documentation”, threejs.org/docs (Accessed: October 2026).

[5] Tailwind Labs, “Tailwind CSS documentation”, tailwindcss.com/docs (Accessed: October 2026).

[6] Vite, “Getting started”, vite.dev/guide (Accessed: October 2026).

[7] DeepSeek, “DeepSeek API documentation”, api-docs.deepseek.com (Accessed: October 2026).

[8] Google, “Progressive web apps”, web.dev/learn/pwa (Accessed: October 2026).

[9] GitHub, “GitHub Pages documentation”, docs.github.com/en/pages (Accessed: October 2026).

[10] Microsoft, “TypeScript handbook”, typescriptlang.org/docs/handbook (Accessed: October 2026).

[11] I. Sommerville, Software Engineering, 10th ed. Boston: Pearson, 2016.

[12] S. Krug, Don’t Make Me Think! A Common Sense Approach to Web Usability, 3rd ed. San Francisco: New Riders, 2014.